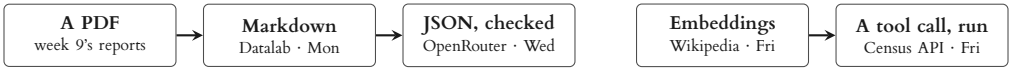
AI APIs as Data Tools

Week 13 · APIs module · Textbook Chapter 13

Brian C. Keegan, Ph.D.
Associate Professor, Department of Information Science
University of Colorado Boulder

November 9, 11 & 13, 2026

This week at a glance



Monday and Wednesday: one pipeline on week 9's Boulder revenue reports. Friday: Wikipedia, and week 11's Census API. No chatbots: each request gets one JSON answer, and we check every answer.

Monday | Lab, part 1: document understanding

- Turn a PDF, pictures of tables too, into Markdown; check it with week 9's sums

Wednesday | Lab, part 2: a model as a function

- Send one page; get back JSON with the fields we ask for; check it against week 9

Friday | Lab, part 3: search by meaning, and tools

- Compare Wikipedia leads by meaning; run a Census call a model wrote
- 15 minutes on portfolio checkpoint 3

Keys: two new, and week 11's

Before Monday: `DATALAB_KEY` from datalab.to, and `OPENROUTER_KEY` from openrouter.ai. Set each as an environment variable, close the terminal, open a new one, and start Jupyter from there. Friday also needs week 11's `CENSUS_KEY`.

Due Sunday, Nov 15

Chapter 13's **Recommended Exercises**, from its companion notebook, and portfolio checkpoint 3, *Analyze your data*. Graded on good-faith attempt to complete rather than getting everything working.

1. Monday · Live-Coding Lab

Daily note questions

- **Question from the daily note?** — Answer, in one line.

This week, every class is live coding

Find it | `labs/` in the course repository

1. Open the **labs** folder of `cuinfoscience/INF04617-Fall2026` on GitHub
2. Open `week-13`, click `lab-13.ipynb`, then **Download raw file**
3. Open it in JupyterLab in the `webdata` environment

Use it | one part a day

- I write the code on screen, and you write it in your copy
- Put **your** email in the User-Agent in Step 0
- Today: Part 1, Steps 0–5, with both keys set
- Wednesday and Friday: download the new copy and start at that part's setup step

Solutions

`lab-13-key.ipynb`, beside it: every cell filled in and run. Fall behind? Copy the cell from the key and keep going.

Nothing to submit

The lab is practice. After each step, a **Discuss** box holds questions we'll talk over before the next.

Error flavors

A `429` ("too many requests") from a free model means too many people are using it: wait, and send it again. A `KeyError` on `"choices"` means the API sent back an error, not an answer: print it and look. A request that fails for everyone is *broken*: tell me, or open an issue.

A model called like a function

A chatbot answers a person. This week a model answers a program: one request, one answer, in a shape we choose. We check the answer before we use it.

Three kinds of AI API, one a day:

- **Document understanding**: a PDF in, Markdown out, pictures of tables included (Datalab)
- **Structured output**: a page in, JSON with the fields we ask for out (OpenRouter)
- **Embeddings**: text in, a list of numbers out, so we can search by meaning (OpenRouter)

Check every answer. You can't tell a model's wrong number from a right one by looking at it.

What you send, and to whom

Every page we send goes to a company: Datalab, or OpenRouter and whoever runs the model. Boulder's reports are public records that name no private individuals. Send nothing you couldn't publish.

Page 1 stays unprinted

Each report's cover gives a city staff member's email and phone. Week 9 blacked it out, and we never print it.

Step 0: two keys

1. **Datalab**: sign up at datalab.to. Copy a key from API Keys. The free plan includes a monthly allowance; this lab spends about 9 cents of it.
2. **OpenRouter**: sign up at openrouter.ai. Create a key at Keys. Models whose ID ends in `:free` cost nothing, up to a daily limit.
3. In a terminal (on Windows, the Anaconda Prompt), store both in the `webdata` environment:

```
conda env config vars set DATALAB_KEY="your-key" -n webdata
conda env config vars set OPENROUTER_KEY="your-key" -n webdata
```

Then close that terminal, open a new one, run `conda activate webdata`, and start Jupyter from it. Datalab's docs call its key `DATALAB_API_KEY`; we name it as we named `CENSUS_KEY`.

```
print("DATALAB_KEY:", "set"
      if os.environ.get("DATALAB_KEY")
      else "MISSING")
print("OPENROUTER_KEY:", "set"
      if os.environ.get("OPENROUTER_KEY")
      else "MISSING")
# DATALAB_KEY: set
# OPENROUTER_KEY: set
```

It's a password

Never paste a key into a notebook cell or a code file. `MISSING`? Quit Jupyter and start it from a new terminal. Or type the key into Step 0's `getpass()` box.

Step 1: submit a document, get a ticket

```
datalab_base = "https://www.datalab.to/api/v1"
book = ("https://raw.githubusercontent.com/cuinfoscience/"
        "Web-Data-Science-Book/main/data/ch-09/")
data = {"file_url": book + "revenue-report-2024-12.pdf",
        "output_format": "markdown",
        "mode": "balanced", "paginate": "true",
        "skip_cache": "true"}
job = requests.post(f"{datalab_base}/convert", data=data,
    headers={"X-API-Key": datalab_key, **HEADERS}).json()
print(job)
# {'success': True, 'error': None,
#  'request_id': 'S8zq-0yiaN7zc1gmv3CUAg',
#  'request_check_url': 'https://www.datalab.to/api/v1/
#      convert/S8zq-0yiaN7zc1gmv3CUAg',
#  'versions': None, 'checkpoint_reused': None}
```

Converting takes seconds, so Datalab first answers with a **ticket**: `request_check_url`, the address where the result will appear.

Docs: Datalab's Convert API

Why `skip_cache`?

Datalab keeps a copy of each file it converts. Send the same file again, and it returns that copy, free. `skip_cache` makes it convert again, so we see the real cost.

We send a link to the PDF

`file_url` points to the book's copy of week 9's PDF, and Datalab fetches it. A file only on your laptop goes as an upload instead.

Step 2: wait for the result

```
check = requests.get(job["request_check_url"],
    headers={"X-API-Key": datalab_key, **HEADERS}).json()
print(check["status"])
# processing

# Given: ask every two seconds until it's done
while check["status"] != "complete":
    time.sleep(2)
    check = requests.get(job["request_check_url"],
        headers={"X-API-Key": datalab_key,
                 **HEADERS}).json()
print(check["page_count"], "pages,", check["total_cost"],
      "cents,", round(check["runtime"], 1),
      "seconds of processing")
# 15 pages, 4.5 cents, 10.1 seconds of processing
```

`status` stays `processing` until it's `complete`. The loop asks until then, as week 12's loop followed the `cursor` until it ran out.

Docs: the result check

What it cost

Measured on November 7, for the chapter:

- All seven year-end reports, `balanced`: 106 pages for 31.8 cents, 12–15 seconds each
- This report in `lite` mode: 0.56 cents, with rougher tables
- The free allowance: about 3,000 pages a month

Discuss

The loop has no limit. If Datalab never finished, what would your notebook do?

Step 3: read the Markdown, page by page

```
markdown = check["markdown"]
pages = re.split(r"\n*\{\d+\}-{10,}\n*",
                 markdown)[1:]
print(len(pages), "pages")
# 15 pages
print(pages[1][:1200])   # the summary

exhibit = pages[14][pages[14].index("|"):]
for line in exhibit.splitlines()[:10]:
    print(line[:150])
```

With `paginate`, each page starts with a marker such as `{14}` and a line of dashes. Split on the markers, and drop the empty piece before the first one.

Never `pages[0]`

The cover gives a staff member's email and phone. So does Exhibit 3's note, above its table. We print from the table's first `|`.

Exhibit 3, a picture of a table, as Markdown

```
| REVENUE<br>CATEGORY              | YEAR        | JAN
|----------------------------------|-------------|------
|                                  |             |
| <b>Retail Sales Tax</b>          | 2019        | 8,590,452
|                                  | 2020        | 7,761,028
|                                  | 2021        | 8,059,343
|                                  | 2022        | 9,592,143
|                                  | 2023        | 10,246,130
...
```

The note beside this table asks readers to phone the city for "a screen readable version". Week 9's pdfplumber stopped here: a picture has no text layer to read. Datalab read all 30 rows of figures.

Chart 8, page 9: a pie chart, figures printed on its slices

Category	Amount
State Share-Back	$529,220
Medical Marijuana (3.86% Base)	$52,937
Recreational Marijuana (3.86% Base)	$1,262,688
Recreational Marijuana (3.5% Additional + 5.0% Excise)	$1,341,007

Under the table, Datalab's description:
"The largest slice is Recreational Marijuana (3.86% Base) at $1,262,688, followed by Recreational Marijuana (3.5% Additional + 5.0% Excise) at $1,341,007."

Datalab **read** the table from the slices. Its model **wrote** the sentence, and the sentence ranks the slices wrongly.

Page 10: a bar chart with no numbers

The sugar-sweetened beverage chart prints bars and an axis, and no figures. Datalab wrote a table beside it anyway: 72 values, one per month for six years, each marked `~`, "about".

December 2024: `~$285,000`. That number appears nowhere in the PDF.

Discuss

What else on these pages did a model write rather than read? How would you tell which is which?

Step 4: check what it read

```
checked = 0
for line in pages[14].splitlines():
    cells = [c.strip().replace("<b>", "")
             .replace("</b>", "").replace(",", "")
             for c in line.split("|")]
    if len(cells) > 16 and re.fullmatch(r"20\d\d", cells[2]):
        months = sum(int(c) for c in cells[3:15])
        checked += 1
        if abs(months - int(cells[15])) > 5:
            print(cells[2], months, cells[15])
print(checked, "rows checked")
# 30 rows checked
```

Week 9's first check: a row's twelve months should add up to its total. Nothing printed, so no row was off by more than $5.

How close

17 of the 30 rows add up exactly; the rest are within $2, because the report rounds each figure to the dollar.

Microsprint: 2018

The same steps on the 2018 report, whose Exhibit 3 is text, not a picture: 15 pages, 4.5 cents, 28 rows checked, all exact.

What a sum can't catch

Week 9's Docling read 21 of these 30 rows whole, and put two of them beside the wrong label. Those rows passed the sums.

Step 5 saves the Markdown to a file, so Wednesday doesn't pay to read it again.

2. Wednesday · Live-Coding Lab

Today: Part 2, a model as a function

We call a language model like any other API. We send an instruction and one page, and get back JSON in a shape we define. Then we check it against week 9.

- Find the page with the summary table's last row
- Ask for its totals as JSON, with a **JSON Schema**: a description of the fields we want and their types
- Check them against pdfplumber's figures; then do all seven reports

Use it | Steps 6–10

- Download the new `lab-13.ipynb`; start at **Step 6**
- Only `OPENROUTER_KEY` today: we read the book's saved Markdown of all seven reports, made as on Monday

Free models are shared

Models whose ID ends in `:free` cost nothing, and our whole class shares them. When one is busy, it answers `429`. We send a list of `models`, and OpenRouter tries the next one.

OpenRouter

One kind of request reaches models from many companies. OpenRouter is also one more company that sees your documents.

Steps 7–8: one page, and the shape of the answer

```
markdown = requests.get(
    saved + "revenue-report-2024-12.md",
    headers=HEADERS).text
pages = re.split(r"\n*\{\d+\}-{10,}\n*",
                 markdown)[1:]
summary = [page for page in pages
           if "Total Sales and Use Tax" in page][0]
```

That one page, 4,902 characters, is all the model sees.

The instruction

"You extract figures from one page of a City of Boulder revenue report. Copy numbers exactly as printed, without $ or commas. Use only the page."

```
# Given: the answer's shape, in JSON Schema
SCHEMA = {
  "type": "object",
  "properties": {
    "month": {"type": "string", "enum": MONTHS},
    "year": {"type": "integer"},
    "total_prior": {"type": "integer",
      "description": "The table's last row, "
        "Total Sales and Use Tax, for the "
        "earlier year, in dollars"},
    "total_current": {"type": "integer", ...},
  },
  "required": ["month", "year",
               "total_prior", "total_current"],
  "additionalProperties": False,
}
```

In the lab, every field has a description too. `enum` lists the twelve months the model may give.

Step 8: one request, one JSON answer

```
MODELS = ["nvidia/nemotron-3.5-super-vl:free",
          "google/gemma-4-31b-it:free",
          "dots-studio/dots-3-note-preview:free"]
body = {"models": MODELS, "temperature": 0,
  "messages": [
    {"role": "system", "content": INSTRUCTION},
    {"role": "user", "content": summary}],
  "response_format": {"type": "json_schema",
    "json_schema": {"name": "totals", "strict": True,
                    "schema": SCHEMA}}}
response = requests.post(
    f"{openrouter_base}/chat/completions", json=body,
    headers={"Authorization": f"Bearer {openrouter_key}",
             **HEADERS}).json()
```

Docs: OpenRouter's structured outputs and model fallbacks; free models checked November 7

Each part of the body

- `messages`: a `system` instruction, then the `user` page. We never add a third message.
- `response_format`: structured output, an answer in `SCHEMA`'s shape. `strict`: no other fields.
- `temperature`: how much randomness the model adds. 0 asks for the model's most likely answer.
- `models`: if the first is busy, try the next. Embeddings get no list (Friday).

Step 9: the answer, and the check

```
print(response["model"])
print(response["usage"])
answer = json.loads(
    response["choices"][0]["message"]["content"])
print(answer)
# nvidia/nemotron-3.5-super-vl:free
# {'prompt_tokens': 1568, 'completion_tokens': 2438, ...
#  'reasoning_tokens': 2385, ...}
# {'month': 'December', 'year': 2025,
#  'total_prior': 175517129, 'total_current': 173908431}

prior, current = TRUTH["revenue-report-2024-12"]
print("prior:  ", answer["total_prior"], prior,
      answer["total_prior"] == prior)
print("current:", answer["total_current"], current,
      answer["total_current"] == current)
print(answer["year"], answer["month"])
# prior:   175517129 175517129 True
# current: 173908431 173908431 True
# 2025 December
```

Right totals, wrong year

Both totals match week 9's. The report is 2024's, but the model said 2025, from "received by the city through January 2025".

`TRUTH` is our **ground truth**, figures we trust: week 9's pdfplumber figures for the last row. Without them, would you have trusted the totals? Would you have noticed the year?

`usage`

A **token** is a word or a piece of one. 1,568 tokens in, 2,438 out. 2,385 of them were `reasoning_tokens`, notes the model writes to itself before it answers: it "thought" for about 2,400 tokens to copy two numbers. Cost: 0.

Non-determinism: the same request, different answers

```
for attempt in range(3):
    response = requests.post(
        f"{openrouter_base}/chat/completions",
        json=body, headers=...).json()
    print(response["model"], json.loads(
        response["choices"][0]["message"]["content"]))
    time.sleep(2)
# nvidia/nemotron-3.5-super-vl:free {'month': 'December',
#  'year': 2025, 'total_prior': 175517129,
#  'total_current': 173908431}
# nvidia/nemotron-3.5-super-vl:free {'month': 'December',
#  'year': 2025, 'total_prior': 175517129,
#  'total_current': 173908431}
# nvidia/nemotron-3.5-super-vl:free {'month': 'December',
#  'year': 2024, 'total_prior': 175517129,
#  'total_current': 173908431}
```

In the key's run: the same totals three times, and two years. The third answer said 2024, the right year, from the same request.

Other runs

- On November 7, the same 2024 request once got **both totals wrong**: 2024's total as 2023's, and the subtotal above it as 2024's
- Step 10's seven reports, both totals right: 4, 5 and 6 of 7 on November 7; 5 of 7 twice on November 9, with different reports wrong
- Runs for the chapter got the year as 2023, then 2024

Same code, same free model, `temperature` 0.

Why answers change

`temperature` 0 asks for the most likely answer. But shared hardware can change the arithmetic slightly, and 2,400 tokens of reasoning give that small change room to grow.

Step 10: seven reports

```
rows = []
for report, (prior, current) in TRUTH.items():
    pages = re.split(r"\n*\{\d+\}-{10,}\n*",
        requests.get(saved + report + ".md",
                     headers=HEADERS).text)[1:]
    summary = [page for page in pages
        if "Total Sales and Use Tax" in page][0]
    answer, model = extract_totals(summary)
    rows.append({"report": report, "model": model,
        **answer,
        "prior_ok": answer["total_prior"] == prior,
        "current_ok":
            answer["total_current"] == current})
    time.sleep(2)
results = pd.DataFrame(rows)
```

`extract_totals(page)` is Step 8's request in a function, because we send it seven times. One model answered all seven.

Checked against week 9

Report	Prior	Current	It said
2018	right	right	December 2018
2019	**wrong**	**wrong**	January 2020
2020	**wrong**	**wrong**	December 2020
2021-12	right	right	December 2021
2022-12	right	right	December 2022
2023-12	right	right	December 2023
2024-12	right	right	December 2025

The lab key's run. Both totals right: 5 of 7.

Every report covers through December of its own year. The month or the year is wrong in two of seven: 2019's and 2024's.

What went wrong: one mistake, two reports

The 2019 report's summary page, its last two rows:

	2018	2019
Total … current economic activity	$139,292,879	$139,548,151
Total Sales and Use Tax	$140,609,751	$141,954,650

The model's answer:
`total_prior`: 141954650 `total_current`: 139548151

For 2019 and 2020, it took the subtotal above the last row as the report year's total, and moved the report year's total into the year before. In this run, 2018 came back right; in an earlier run the same day, it was one of the two wrong.

The total row was there

Both pages hold the whole table, the total row included. The model had the right figures in front of it.

The wrong month and year

The 2019 report's "January 2020" comes from its sentence "received by the city through January 2020". The 2024 report's 2025 comes from the same sentence.

Discuss

A better model, a better instruction, or a better page: which would you try first, and how would you know it worked?

3. Friday · Live-Coding Lab

Today: Part 3, search by meaning, and tools

On Wednesday the model read one page that we chose. With a hundred passages, we need a way to find the ones worth reading, and to say how far apart two of them are.

An **embedding** turns a passage into a list of numbers, so that passages about the same thing get similar numbers. To an embedding, every year's marijuana page in Boulder's reports looks alike, whatever figures it prints. So today we use prose that editors keep rewriting: Wikipedia.

Lab | Steps 11–15, in the new `lab-13.ipynb`

- The **lead**, the paragraphs above an article's first heading, of five articles, on January 1 of each year from 2004 to 2026
- How close each year's lead is to today's; an answer from the leads, with sources
- A model writes one request to week 11's Census API, and our code runs it
- Last 15 minutes: portfolio checkpoint 3

Two keys today

`OPENROUTER_KEY`, and week 11's `CENSUS_KEY` for Step 14. If week 11's lab could read it, so can this one. `MISSING`? Run Step 11's given `getpass()` cell.

Five articles

Barack Obama, Hillary Clinton, Donald Trump, Joe Biden and Kamala Harris. Every version of an article is kept in its history (week 10), so we can read each lead as it stood on New Year's Day.

Chapter 13, "Embeddings and Retrieval", "Retrieval-Augmented Extraction", and "Tools, MCP, and Skills".

Step 11: one old version, by hand

```
API = "https://en.wikipedia.org/w/api.php"
params = {"action": "query", "titles": "Barack Obama",
    "prop": "revisions", "rvprop": "ids|timestamp",
    "rvlimit": 1, "rvstart": "2009-01-01T00:00:00Z",
    "rvdir": "older", "format": "json"}
data = requests.get(API, params=params,
                    headers=HEADERS).json()
page = list(data["query"]["pages"].values())[0]
revision = page["revisions"][0]
print(revision)
# {'revid': 261157071, 'parentid': 261086351,
#  'timestamp': '2008-12-31T20:44:09Z'}

params = {"action": "parse", "oldid": revision["revid"],
          "prop": "text", "section": 0,
          "format": "json", "formatversion": 2}
html = requests.get(API, params=params,
    headers=HEADERS).json()["parse"]["text"]
```

Docs: the Action API's revisions and parse

Two requests

The first finds the newest revision at or before midnight UTC, January 1, 2009: start at `rvstart`, walk back in time, and stop after one. That's the version a reader saw on New Year's Day. Its `revid` names it for good.

The second, `action=parse`, returns that version as HTML. `section=0` asks for the lead alone.

Why the lead?

The embedding model reads at most 4,096 tokens of a text. On January 1, 2009, Obama's whole article was already 135,666 bytes of wikitext. The lead fits, and its job is to summarize the article.

Step 11: today's templates in a 2009 lead

```
soup = BeautifulSoup(html, "html.parser")
for paragraph in soup.find_all("p"):
    print(paragraph.get_text(" ", strip=True)[:150])
# Template:Active editnotice Template:Redirect4 ...
# Template:FixHTML
# Barack Hussein Obama II ( Template:Pron-en ; born
#   August 4, 1961) is an American politician and ...

for tag in soup.select("table, .reference, .hatnote, "
        ".shortdescription, style, "
        "a.new[title^='Template:'], .error"):
    tag.decompose()
paragraphs = []
for paragraph in soup.find_all("p"):
    if paragraph.get_text(strip=True):
        paragraphs.append(
            paragraph.get_text(" ", strip=True))
lead = re.sub(r" ([,.;:])", r"\1",
              "\n\n".join(paragraphs))
print(lead[:300])
# Barack Hussein Obama II (; born August 4, 1961) is
#   an American politician and the President-elect ...
```

Leftovers

Wikipedia keeps an old version's wikitext, not its HTML. So `parse` draws 2009's text with today's templates, and templates deleted since come back as links named `Template:…` An embedding model would read them as part of the lead.

`select()` finds them by CSS selector (week 5), with the tables, footnote marks and error messages, and `decompose()` deletes each. This is the chapter's `lead_text()`.

Then the saved leads

A given cell loads all 116, collected and cleaned this way on November 9. Collecting them drew `429`s for about 30 minutes, at one request a second. Our 2009 lead matches the saved one: `True`.

Discuss: what in a lead might come from today rather than from 2009, without looking broken?

Step 12: embed, and compare each year with today

```
EMBEDDER = "nvidia/nemotron-3-embed-1b:free"
one = requests.post(f"{openrouter_base}/embeddings",
    json={"model": EMBEDDER,
          "input": [leads["lead"][0]]},
    headers={"Authorization": f"Bearer {openrouter_key}",
             **HEADERS}).json()
print(len(one["data"][0]["embedding"]), "numbers")
# 2048 numbers

# Given: embed all 116, 20 to a request, and scale
# each vector to length 1
# (116, 2048)

today = {}
for i, row in leads.iterrows():
    if row["year"] == "current":
        today[row["title"]] = vectors[i]
leads["similarity"] = [vectors[i] @ today[title]
    for i, title in enumerate(leads["title"])]
```

Docs: OpenRouter's embeddings endpoint

Vectors

Each lead comes back as a **vector**, a list of 2,048 numbers. None of them means anything alone. What carries meaning is how two vectors compare.

Cosine similarity

It measures how nearly two vectors point the same way: 1 if exactly, less the further apart they point. For vectors of length 1, it's their **dot product**, `@`: multiply the two lists number by number, and add up.

One embedding model

Two models turn the same text into different numbers, and their vectors can't be compared. So `EMBEDDER` gets no fallback list. If its daily limit stops you, a given cell loads the vectors saved on November 9, from the same model.

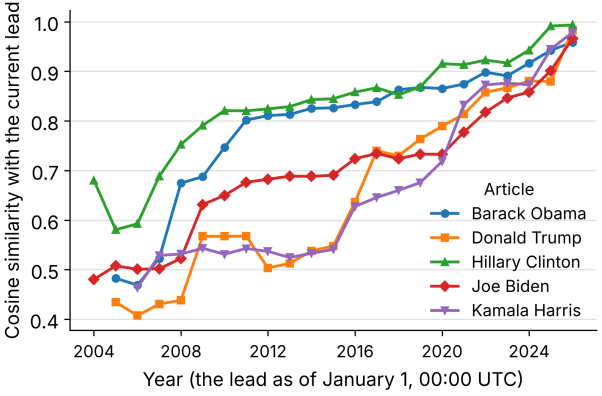
Step 12: twenty-three years of leads

```
table = leads[leads["year"]
    != "current"].pivot(index="year",
    columns="title", values="similarity")
table.index = table.index.astype(int)
fig, ax = plt.subplots(figsize=(8, 4.5))
table.plot(ax=ax, marker="o")
```

Every line climbs toward 1: the closer a January is to today, the more of today's lead it already holds.

The climbs come in steps, and most sit at an election. Trump's rises from 0.637 to 0.740 between 2016 and 2017, when his lead first called him president-elect. Harris's rises from 0.718 to 0.832 between 2020 and 2021.

No line reaches 1, even in 2026. Clinton's comes closest, at 0.993.



Cosine similarity of each January 1 lead to the article's current lead. Drawn from the lab's saved vectors, as Step 12 computes it; the same data as chapter 13's "Similarity Over Time".

Microsprint: the biggest jump

```
title = "Kamala Harris"
jumps = table[title].diff()
year = jumps.idxmax()
print(year, round(jumps.max(), 3))
for y in [year - 1, year]:
    print(y, leads[(leads["title"] == title)
        & (leads["year"] == str(y))]["lead"].iloc[0][:400])
# 2021 0.114
# 2020 Kamala Devi Harris (...; born October 20, 1964)
#   is an American lawyer and politician serving as
#   the junior United States Senator from California
#   since 2017. ...
# 2021 Kamala Devi Harris (...; born October 20, 1964)
#   is an American politician who is the vice
#   president-elect of the United States and the
#   junior United States senator from California. ...
```

`diff()` gives each year's change from the year before, and `idxmax()` the year of the biggest rise.

What a jump can tell you

Harris's lead jumped most in 2021, when it first called her vice president-elect. Biden's jumped most in 2009 (+0.109), for the same reason.

What it can't

Trump's biggest jump (+0.129) came in 2009, with no new office: his lead grew from 93 words to 353. Clinton's went from 0.943 in 2024 to 0.992 in 2025, when editors rewrote her opening sentence.

A similarity measures how far a lead's words moved toward today's. Why they moved is in the text, and in the edit summaries of week 10's revision histories.

Discuss: which claims about how Wikipedia described these people can a similarity not support?

Step 13: find three leads, then ask

```
def search(question, title):
    response = requests.post(
        f"{openrouter_base}/embeddings",
        json={"model": EMBEDDER, "input": [question]},
        headers=...).json()
    query = np.array(response["data"][0]["embedding"])
    leads["score"] = vectors @ (
        query / np.linalg.norm(query))
    return leads[leads["title"] == title
                 ].nlargest(3, "score")

question = ("In which year did the article first say "
    "Joe Biden is the 46th president of the "
    "United States?")
top = search(question, "Joe Biden")
print(top[["id", "score"]])
#                    id     score
# 92     Joe Biden 2026  0.449356
# 90     Joe Biden 2024  0.448661
# 93  Joe Biden current  0.446402
passages = "\n\n".join(f"{row.id}: {row.lead}"
                       for row in top.itertuples())
```

Finding the passages is **retrieval**. Writing an answer from them is generation. Together: **retrieval-augmented generation**, or RAG. A question is embedded like a lead, and the closest leads are the ones the model reads.

`search()` keeps one article's best three, so a question about Biden can't be answered from Harris's article.

```
# Given: an answer with its sources
ANSWER_SCHEMA = {"type": "object",
  "properties": {
    "year": {"type": ["integer", "null"]},
    "source_ids": {"type": "array",
        "items": {"type": "string"}}},
  "required": ["year", "source_ids"],
  "additionalProperties": False}
```

`year` may be **null**, JSON's word for "no value": the model's way to say the passages don't state it. In the lab, each field has a description too.

Step 13: the answer, checked by reading

```
body = {"models": MODELS, "temperature": 0,
  "messages": [
    {"role": "system", "content": ANSWER_INSTRUCTION},
    {"role": "user",
     "content": f"Question: {question}\n\n{passages}"}],
  "response_format": {"type": "json_schema",
    "json_schema": {"name": "answer", "strict": True,
                    "schema": ANSWER_SCHEMA}}}
response = requests.post(
    f"{openrouter_base}/chat/completions",
    json=body, headers=...).json()
print(response["model"], json.loads(
    response["choices"][0]["message"]["content"]))
# nvidia/nemotron-3.5-super-v1:free
# {'year': 2024, 'source_ids': ['Joe Biden 2024']}
```

Wednesday's request, with a new instruction and schema, and three leads in place of a page.

Check it by reading

```
for row in leads[(leads["title"] == "Joe Biden")
        & (leads["year"] >= "2020")].itertuples():
    print(row.id, "|", row.lead[:300], "\n")
# Joe Biden 2021 | ... the president-elect of
#   the United States. Biden defeated incumbent
#   president Donald Trump in the 2020
#   presidential election and will be
#   inaugurated as the 46th president on
#   January 20, 2021. ...
# Joe Biden 2022 | ... who is the 46th and
#   current president of the United States. ...
```

The model was right about the passages it got: 2024 is the earliest of 2026, 2024 and today. It's wrong about the question. 2022 and 2023 say the same, and they never reached the model.

Reading gives **2022**, the first January 1 on which he is the president. A ranking by similarity has no reason to favor the earliest year.

Searching for the words, and a null

A text search

The first tool you'd try is a search for the words, as week 9 searched a PDF's text layer. `leads["lead"].str.contains("46th president")` finds Biden's leads of 2021, 2026 and today.

It misses 2022 to 2025, which call him "the 46th and current president". And its 2021 hit says he "will be inaugurated as the 46th president". Trump's 2025 lead says the same of the 47th.

A search finds only the wording you thought to type, and it can't tell "will be" from "is". Embeddings found leads about the right subject, and missed the first year. The year came from reading.

Microsprint: Harris, the 47th president?

```
question = ("In which year did the article "
    "first say Kamala Harris is the 47th "
    "president of the United States?")
top = search(question, "Kamala Harris")
#                       id     score
# 111  Kamala Harris 2023  0.426085
# 110  Kamala Harris 2022  0.423550
# 112  Kamala Harris 2024  0.423322
# nvidia/nemotron-3.5-super-vl:free
# {'year': None, 'source_ids': []}
```

Null is right: of all 116 leads, only Trump's from 2025 on contain "47th president" (chapter 13). But the model saw three of Harris's 22 leads. Its null says only that those three don't state it.

Discuss: to answer a question about the *first* time, what would you send the model instead of the best three?

Step 14: tools, and the Census Bureau's MCP server

A model can't fetch data: all it can do is write text. Ask one for Boulder County's median household income, and it answers from its training text, with no table or year you can trace.

But a request is text. Tell a model which requests it may make, and it can write one for your program to send. Each such request is a **tool**: a name, a description, and its inputs, as a JSON Schema like Wednesday's.

The **Model Context Protocol** (MCP) is a shared way for a server to say which tools it offers. A program sends the method `tools/list` as **JSON-RPC**, JSON that names a `method` to run, with an `id` the answer repeats.

Docs: the MCP specification, tools; the Census Bureau's Data API MCP server

```
cmcp = ("https://raw.githubusercontent.com/"
    "cuinfoscience/INF04617-Fall2026/main/"
    "labs/week-13/data/census-mcp/")
tools = requests.get(cmcp + "tools.json",
                     headers=HEADERS).json()
for tool in tools:
    print(tool["name"],
          tool["inputSchema"]["required"])
# fetch-aggregate-data ['dataset', 'year', 'get']
# fetch-dataset-geography ['dataset']
# list-datasets []
# resolve-geography-fips ['geography_name']
# search-data-tables []
```

The server runs on your own computer, with Docker or Node and a database it builds. So it was run once, on November 9, and its real tool list and one session were saved for us.

Step 14: the model writes one tool call

```
functions = [{"type": "function", "function": {
    "name": tool["name"],
    "description": tool["description"],
    "parameters": tool["inputSchema"]}}
    for tool in tools]
body = {"models": MODELS, "temperature": 0,
  "tools": functions, "tool_choice": "required",
  "messages": [
    {"role": "system", "content": TOOL_INSTRUCTION},
    {"role": "user", "content": question}]}
response = requests.post(
    f"{openrouter_base}/chat/completions",
    json=body, headers=...).json()
call = response["choices"][0]["message"][
    "tool_calls"][0]["function"]
args = json.loads(call["arguments"])
print(response["model"], call["name"], args)
# nvidia/nemotron-3.5-super-vl:free fetch-aggregate-data
# {'dataset': 'acs/acs5', 'year': 2024,
#  'get': {'group': 'B19013'}, 'for': 'county:013',
#  'in': 'state:08', 'descriptive': True}
```

Week 11's question

"What was the median household income of Boulder County, Colorado (state FIPS 08, county FIPS 013), in the 2024 ACS 5-year estimates, table B19013?"

Each tool's `inputSchema` becomes its `parameters`. `tool_choice` `"required"` asks for a tool call and no text.

The model chose `fetch-aggregate-data`, and asked for `group` B19013, the whole table. In a test the same day, it asked for `B19013_001E` alone, without the margin of error.

Nothing has gone to the Census Bureau yet. The call is a request the model wrote.

Discuss: `inputSchema` and `response_format`: which constrains what a model sends, and which what it returns?

Step 14: run the call yourself

```
url = ("https://api.census.gov/data/"
       f"{args['year']}/{args['dataset']}")
if "group" in args["get"]:
    get = f"group({args['get']['group']})"
else:
    get = ",".join(args["get"]["variables"])
params = {"get": get, "for": args["for"],
          "in": args["in"], "key": census_key}
data = requests.get(url, params=params,
                    headers=HEADERS).json()
print(data)
# [['B19013_001E', 'B19013_001EA', 'B19013_001M',
#   'B19013_001MA', 'GEO_ID', 'NAME', 'state', 'county'],
#  ['103994', None, '2268', None, '0500000US08013',
#   'Boulder County, Colorado', '08', '013']]
row = dict(zip(data[0], data[1]))
print(row["B19013_001E"], row["B19013_001E"] == "103994",
      row.get("B19013_001M"))
# 103994 True 2268
```

Our code does what the server would have done: it turns the model's arguments into week 11's request, and sends it with `census_key`.

Print the data, never the URL

The URL holds your key. The real server reports its URL with the key replaced by `REDACTED`.

The same answer, three ways

$103,994, with a margin of error of $2,268: from this request, from week 11, and from the recorded server's own call:

```
Response from acs/acs5:
NAME: Boulder County, Colorado,
  B19013_001E: 103994, B19013_001M: 2268,
  state: 08, county: 013
```

Every figure came from the Census Bureau's API. The model chose the tool and filled in its arguments.

What your code still decides

Microsprint: ask by name

```
body["messages"][1]["content"] = (
    "What was the median household income of the "
    "city of Boulder, Colorado, in the 2024 ACS "
    "5-year estimates, table B19013?")
response = requests.post(
    f"{openrouter_base}/chat/completions",
    json=body, headers=...).json()
call = response["choices"][0]["message"][
    "tool_calls"][0]["function"]
print(response["model"], call["name"],
      json.loads(call["arguments"]))
# nvidia/nemotron-3.5-super-v1:free
#   resolve-geography-fips
# {'geography_name': 'Boulder city, Colorado',
#  'summary_level': 'Place'}
```

Without the codes, the model chose `resolve-geography-fips`, to turn the name into codes. The real server answers it from its own database. Our code runs only `fetch-aggregate-data`, so it can't run this call.

Which tools run

A model can choose any tool it's shown. A program shown a call it can't run has to say so, and refuse or stop.

Which arguments go out

The model's arguments went into our request unchecked, and our cell quietly dropped one, `descriptive`. They decide what your code asks for, with your key.

Whether the answer is right

MCP fixes how a tool is described and called. We know $103,994 is right because week 11 found it.

Discuss: with week 11's place code for Boulder, 07850, what would you send instead? Who should check the arguments: the model, the schema, our code, or the Census API? (Week 11's API answered `400` to a place without a state.)

Step 14: a skill is a Markdown file

```
skill = requests.get("https://raw.githubusercontent.com/"
    "anthropics/skills/main/skills/pdf/SKILL.md",
    headers=HEADERS).text
print("\n".join(skill.splitlines()[:5]))
# ---
# name: pdf
# description: Use this skill whenever the user wants
#   to do anything with PDF files. This includes reading
#   or extracting text/tables from PDFs, ...
# license: Complete terms in LICENSE.txt
# ---
print([line for line in skill.splitlines()
       if line.startswith("### ")])
# ['### pypdf - Basic Operations',
#  '### pdfplumber - Text and Table Extraction',
#  '### reportlab - Create PDFs', ...]
```

The description goes on to merging, splitting, forms and OCR (reading text from images); we cut it here. Ten `###` headings in all.

Front matter, then instructions

A **skill** is a folder of instructions for a model, plainer still than a tool; `SKILL.md` is its main file. The front matter, between the `---` lines, gives a `name` and `description`. Like a tool's description, they tell a model when to read the rest: 314 lines of Markdown and code.

Its headings name the tools it teaches: **pypdf** and **pdfplumber**, week 9's tools, then reportlab, pdftotext, qpdf, and more.

Discuss

If a model followed this skill on Boulder's reports, which of week 9's checks would still be yours to run?

Step 15: what did we learn, and what's due

Answer as comments

- Datalab read a picture of a table and wrote descriptions of charts. Which of its output would you publish, and which would you check first?
- Which reports did the model get wrong on Wednesday? Was the problem the model, or what we sent it?
- Which of Step 12's jumps could you explain only by reading the leads? What can a similarity not tell you?
- What would you check your own project's model outputs against?

Due Sunday | Recommended Exercises

From chapter 13's companion notebook, on a public PDF you choose, with no private individuals' names:

1. Convert it: Monday's Steps 1–3
2. Write a schema of your own, with fields your document prints
3. Extract: Step 8's request, and `extract_totals()` for more pages
4. Check it: a sum, and a ground truth you find
5. Retrieve and answer with sources; ask one question your document can't answer
6. Report the costs: pages, cents, tokens

Our leads were short enough to embed whole. A document's pages may first need cutting into chunks, as the chapter shows. Graded on good-faith attempt to complete rather than getting everything working.

Friday's last 15 minutes: portfolio checkpoint 3

Due Sunday, Nov 15 | Analyze your data

- **An analysis notebook** that runs top to bottom, with its output visible
- **Results that answer at least part of your question**: the counts or comparisons your proposal named
- **At least one figure** saved in your repository, with a title, labeled axes, and a one-sentence caption
- **The README's Methods section** and your preliminary **Findings**

Run your analysis notebook once, top to bottom, here, where you can ask. Used a model anywhere? Say in Methods which one, when, and what you checked its output against.

Next week: automating data collection (chapter 14).

Key takeaways

1. Call a model **as a function**: one request in, JSON in your schema or one tool call out. Your code decides what runs.
2. **Valid JSON can still be wrong.** Check it against something you trust: a sum, a second tool, a ground truth.
3. Document APIs **read** some things and **write** others. A model wrote the chart's description; the document didn't print it.
4. The same request can give a different answer. **Record** the model, the date, and the raw response.
5. Embeddings measure how alike two texts are. **Read** them to learn why they changed. An answer needs its **sources**, or null.